Graph Data Structure Study Guide

1. Shortest Path in Undirected Graph with Unit Weight

Overview

When all edges in an undirected graph have the same weight (unit weight), we can use **Breadth-First Search (BFS)** to find the shortest path between any two vertices. BFS guarantees the shortest path in unweighted graphs because it explores vertices level by level.

Algorithm Steps

1. Start from the source vertex
2. Use a queue to store vertices to be processed
3. Use a distance array to track the shortest distance from source
4. Use a parent array to reconstruct the path
5. Process each vertex exactly once

Java Implementation

java

```java
import java.util.*;

class Graph {
    private int vertices;
    private ArrayList<ArrayList<Integer>> adjList;

    public Graph(int v) {
        vertices = v;
        adjList = new ArrayList<>(v);
        for (int i = 0; i < v; i++) {
            adjList.add(new ArrayList<>());
        }
    }

    public void addEdge(int u, int v) {
        adjList.get(u).add(v);
        adjList.get(v).add(u); // Undirected graph
    }

    public List<Integer> shortestPath(int src, int dest) {
        int[] distance = new int[vertices];
        int[] parent = new int[vertices];
        boolean[] visited = new boolean[vertices];

        Arrays.fill(distance, -1);
        Arrays.fill(parent, -1);

        Queue<Integer> queue = new LinkedList<>();
        queue.offer(src);
        visited[src] = true;
        distance[src] = 0;

        while (!queue.isEmpty()) {
            int current = queue.poll();

            if (current == dest) {
                break;
            }

            for (int neighbor : adjList.get(current)) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    distance[neighbor] = distance[current] + 1;
                    parent[neighbor] = current;
                    queue.offer(neighbor);
                }
```

```
            }
        }

        // Reconstruct path
        List<Integer> path = new ArrayList<>();
        if (distance[dest] == -1) {
            return path; // No path exists
        }

        int current = dest;
        while (current != -1) {
            path.add(current);
            current = parent[current];
        }
        Collections.reverse(path);
        return path;
    }

    public int getShortestDistance(int src, int dest) {
        List<Integer> path = shortestPath(src, dest);
        return path.isEmpty() ? -1 : path.size() - 1;
    }
}
```

Time and Space Complexity

- **Time Complexity**: O(V + E) where V = vertices, E = edges
- **Space Complexity**: O(V) for the queue and arrays

2. Dijkstra's Algorithm Using Priority Queue

Overview

Dijkstra's algorithm finds the shortest path from a source vertex to all other vertices in a weighted graph with non-negative edge weights. Using a priority queue (min-heap) optimizes the algorithm by always processing the vertex with the minimum distance first.

Algorithm Steps

1. Initialize distances to all vertices as infinity, except source (distance = 0)
2. Add source vertex to priority queue
3. While priority queue is not empty:
    - Extract vertex with minimum distance
    - Update distances of all adjacent vertices if a shorter path is found
    - Add updated vertices to priority queue

Java Implementation

java

```java
import java.util.*;

class Edge {
    int destination;
    int weight;

    public Edge(int dest, int w) {
        destination = dest;
        weight = w;
    }
}

class Node implements Comparable<Node> {
    int vertex;
    int distance;

    public Node(int v, int d) {
        vertex = v;
        distance = d;
    }

    @Override
    public int compareTo(Node other) {
        return Integer.compare(this.distance, other.distance);
    }
}

class WeightedGraph {
    private int vertices;
    private ArrayList<ArrayList<Edge>> adjList;

    public WeightedGraph(int v) {
        vertices = v;
        adjList = new ArrayList<>(v);
        for (int i = 0; i < v; i++) {
            adjList.add(new ArrayList<>());
        }
    }

    public void addEdge(int src, int dest, int weight) {
        adjList.get(src).add(new Edge(dest, weight));
        adjList.get(dest).add(new Edge(src, weight)); // For undirected graph
    }

    public int[] dijkstraPriorityQueue(int source) {
        int[] distance = new int[vertices];
```

```java
        boolean[] visited = new boolean[vertices];

        Arrays.fill(distance, Integer.MAX_VALUE);
        distance[source] = 0;

        PriorityQueue<Node> pq = new PriorityQueue<>();
        pq.offer(new Node(source, 0));

        while (!pq.isEmpty()) {
            Node current = pq.poll();
            int u = current.vertex;

            if (visited[u]) continue;
            visited[u] = true;

            for (Edge edge : adjList.get(u)) {
                int v = edge.destination;
                int weight = edge.weight;

                if (!visited[v] && distance[u] + weight < distance[v]) {
                    distance[v] = distance[u] + weight;
                    pq.offer(new Node(v, distance[v]));
                }
            }
        }

        return distance;
    }

    public void printShortestPaths(int source) {
        int[] distances = dijkstraPriorityQueue(source);

        System.out.println("Shortest distances from vertex " + source + ":");
        for (int i = 0; i < vertices; i++) {
            if (distances[i] == Integer.MAX_VALUE) {
                System.out.println("Vertex " + i + ": No path");
            } else {
                System.out.println("Vertex " + i + ": " + distances[i]);
            }
        }
    }
}
```

Time and Space Complexity

- **Time Complexity**: O((V + E) log V) where V = vertices, E = edges

- **Space Complexity**: O(V) for the priority queue and arrays

3. Dijkstra's Algorithm Using Set

Overview

An alternative implementation of Dijkstra's algorithm uses a TreeSet (or any balanced binary search tree) instead of a priority queue. This approach allows for efficient removal of elements and can be more intuitive in certain scenarios.

Algorithm Steps

1. Initialize distances to all vertices as infinity, except source (distance = 0)
2. Add all vertices to a set with their distances
3. While set is not empty:
    - Extract vertex with minimum distance from set
    - Update distances of adjacent vertices
    - Remove old entries and add updated entries to set

Java Implementation

java

```java
import java.util.*;

class VertexDistance implements Comparable<VertexDistance> {
    int vertex;
    int distance;

    public VertexDistance(int v, int d) {
        vertex = v;
        distance = d;
    }

    @Override
    public int compareTo(VertexDistance other) {
        if (this.distance != other.distance) {
            return Integer.compare(this.distance, other.distance);
        }
        return Integer.compare(this.vertex, other.vertex);
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        VertexDistance that = (VertexDistance) obj;
        return vertex == that.vertex && distance == that.distance;
    }

    @Override
    public int hashCode() {
        return Objects.hash(vertex, distance);
    }
}

class WeightedGraphSet {
    private int vertices;
    private ArrayList<ArrayList<Edge>> adjList;

    public WeightedGraphSet(int v) {
        vertices = v;
        adjList = new ArrayList<>(v);
        for (int i = 0; i < v; i++) {
            adjList.add(new ArrayList<>());
        }
    }

    public void addEdge(int src, int dest, int weight) {
```

```java
        adjList.get(src).add(new Edge(dest, weight));
        adjList.get(dest).add(new Edge(src, weight)); // For undirected graph
    }

    public int[] dijkstraSet(int source) {
        int[] distance = new int[vertices];
        Arrays.fill(distance, Integer.MAX_VALUE);
        distance[source] = 0;

        TreeSet<VertexDistance> set = new TreeSet<>();

        // Add all vertices to set
        for (int i = 0; i < vertices; i++) {
            set.add(new VertexDistance(i, distance[i]));
        }

        while (!set.isEmpty()) {
            VertexDistance current = set.pollFirst();
            int u = current.vertex;

            for (Edge edge : adjList.get(u)) {
                int v = edge.destination;
                int weight = edge.weight;

                if (distance[u] + weight < distance[v]) {
                    // Remove old entry
                    set.remove(new VertexDistance(v, distance[v]));

                    // Update distance
                    distance[v] = distance[u] + weight;

                    // Add updated entry
                    set.add(new VertexDistance(v, distance[v]));
                }
            }
        }

        return distance;
    }

    public List<Integer> getShortestPath(int source, int destination) {
        int[] distance = new int[vertices];
        int[] parent = new int[vertices];

        Arrays.fill(distance, Integer.MAX_VALUE);
        Arrays.fill(parent, -1);
        distance[source] = 0;
```

```java
        TreeSet<VertexDistance> set = new TreeSet<>();

        for (int i = 0; i < vertices; i++) {
            set.add(new VertexDistance(i, distance[i]));
        }

        while (!set.isEmpty()) {
            VertexDistance current = set.pollFirst();
            int u = current.vertex;

            if (u == destination) break;

            for (Edge edge : adjList.get(u)) {
                int v = edge.destination;
                int weight = edge.weight;

                if (distance[u] + weight < distance[v]) {
                    set.remove(new VertexDistance(v, distance[v]));
                    distance[v] = distance[u] + weight;
                    parent[v] = u;
                    set.add(new VertexDistance(v, distance[v]));
                }
            }
        }

        // Reconstruct path
        List<Integer> path = new ArrayList<>();
        if (distance[destination] == Integer.MAX_VALUE) {
            return path; // No path exists
        }

        int current = destination;
        while (current != -1) {
            path.add(current);
            current = parent[current];
        }
        Collections.reverse(path);
        return path;
    }
}
```

Time and Space Complexity

- **Time Complexity**: $O(V^2 \log V)$ in worst case, but often performs better in practice
- **Space Complexity**: $O(V)$ for the set and arrays

Comparison: Priority Queue vs Set

- **Priority Queue**: Better for sparse graphs, allows duplicate entries
- **Set**: Better for dense graphs, no duplicate entries, allows efficient removal
- **Memory**: Set approach uses slightly more memory due to TreeSet overhead
- **Performance**: Priority queue generally performs better for most practical cases

---

Quiz Questions

1. What is the time complexity of finding shortest path in an unweighted graph using BFS? a) $O(V^2)$ b) O(V + E) c) O(V log V) d) O(E log V)

Correct Answer: b) O(V + E)

2. Which data structure is used in the optimized version of Dijkstra's algorithm? a) Stack b) Queue c) Priority Queue (Min-Heap) d) Linked List

Correct Answer: c) Priority Queue (Min-Heap)

3. What is the main requirement for Dijkstra's algorithm to work correctly? a) Graph must be connected b) All edge weights must be positive c) Graph must be acyclic d) Graph must be undirected

Correct Answer: b) All edge weights must be positive

4. In the Set-based implementation of Dijkstra's algorithm, what happens when we find a shorter path to a vertex? a) We ignore the update b) We add a new entry to the set c) We remove the old entry and add the updated entry d) We clear the entire set

Correct Answer: c) We remove the old entry and add the updated entry

5. What is the space complexity of BFS for shortest path in unweighted graphs? a) O(1) b) O(V) c) O(E) d) O(V + E)

Correct Answer: b) O(V)

Practice Coding Questions

1. Shortest Path Between Two Nodes Write a Java method that takes an unweighted undirected graph, a source vertex, and a destination vertex, and returns the length of the shortest path between them. If no path exists, return -1.

2. All Pairs Shortest Path Implement a method that finds the shortest distance between every pair of vertices in a weighted graph using Dijkstra's algorithm. Return a 2D array where result[i][j] represents the shortest distance from vertex i to vertex j.

3. Path Reconstruction Modify the Dijkstra's algorithm implementation to not only return the shortest distances but also the actual shortest paths from the source to all other vertices. Return a map where the key is the destination vertex and the value is a list representing the shortest path.

4. K Shortest Paths Implement a method that finds the K shortest paths from a source vertex to a destination vertex in a weighted graph. Return a list of paths, where each path is represented as a list of vertices.

5. Shortest Path with Obstacles Given a 2D grid where 0 represents an empty cell and 1 represents an obstacle, write a method to find the shortest path from the top-left corner to the bottom-right corner. You can only move in four directions (up, down, left, right).